

D&D 3.5e Magic Item Enchantment System — Implementation Plan

For Unity Prototype at `/home/ubuntu/dnd35prototype`

Table of Contents

1. [Architecture Overview](#)
 2. [File Structure](#)
 3. [Data Structures](#)
 4. [EnchantmentProperties — Central Database](#)
 5. [ItemEnchantment — Component Model](#)
 6. [EnchantmentFactory — Variant Generation](#)
 7. [EnchantmentEffects — Runtime Combat Integration](#)
 8. [Price Calculation System](#)
 9. [Validation System](#)
 10. [Loot Generation](#)
 11. [Combat Integration Points](#)
 12. [Existing Codebase Integration](#)
 13. [Implementation Schedule](#)
 14. [Testing Plan](#)
-

1. Architecture Overview

Design Principles (Same as Material System)

1. **Central Enchantment Properties** — All enchantment stats defined in one place (`EnchantmentProperties.cs`)
2. **Dynamic Runtime Calculations** — Bonuses computed at access time, not baked into item data
3. **Factory Pattern** — Generate enchanted variants programmatically
4. **No Hardcoding** — Adding a new enchantment = add enum + add properties entry. Zero changes to combat/UI code.
5. **Component Model** — Multiple enchantments per item via a list, not individual booleans

Data Flow



Relationship to Existing Systems

Material System (EXISTING)

MaterialProperties.cs

ItemMaterial.cs

ItemMaterialFactory.cs



ItemData.Material

ItemData.IsMasterwork



Effective* properties
(ACP, MaxDex, Weight, etc.)

Enchantment System (NEW)

EnchantmentProperties.cs

ItemEnchantment.cs (enum + data)

EnchantmentFactory.cs

EnchantmentEffects.cs (combat)



ItemData.Enchantments (List)

ItemData.EnchantmentBonus (existing)



TotalBonusEquivalent, EnchantedPrice
(computed at runtime)

2. File Structure

Assets/Scripts/Equipment/

ItemMaterial.cs

MaterialProperties.cs

ItemMaterialFactory.cs

MATERIAL_SYSTEM_ARCHITECTURE.md

ItemEnchantment.cs

class

EnchantmentProperties.cs
source of truth)

EnchantmentFactory.cs

EnchantmentEffects.cs

EnchantmentValidator.cs

prereqs)

ENCHANTMENT_SYSTEM_ARCHITECTURE.md # NEW - Documentation

EXISTING - enum + material data class

EXISTING - material stats database

EXISTING - material variant factory

EXISTING - material docs

NEW - EnchantmentType enum + EnchantmentData

NEW - Central enchantment database (single

NEW - Create enchanted item variants

NEW - Runtime combat effect application

NEW - Validation rules (compatibility, cap,

3. Data Structures

3a. EnchantmentType Enum

```

/// <summary>
/// All weapon, armor, and shield special abilities from DMG 3.5e.
/// Organized by category for clarity; enum value order doesn't matter.
/// </summary>
public enum EnchantmentType
{
    // =====
    // WEAPON – Elemental Damage (+1d6 per hit)
    // =====
    Flaming,           // +1 equiv, 1d6 fire
    Frost,             // +1 equiv, 1d6 cold
    Shock,             // +1 equiv, 1d6 electricity

    // =====
    // WEAPON – Elemental Burst (1d6 + crit burst)
    // =====
    FlamingBurst,      // +2 equiv, 1d6 fire + 1d10 on crit
    IcyBurst,          // +2 equiv, 1d6 cold + 1d10 on crit
    ShockingBurst,     // +2 equiv, 1d6 elec + 1d10 on crit

    // =====
    // WEAPON – Alignment Damage (+2d6 vs alignment)
    // =====
    Holy,              // +2 equiv, 2d6 vs evil
    Unholy,            // +2 equiv, 2d6 vs good
    Anarchic,          // +2 equiv, 2d6 vs lawful
    Axiomatic,         // +2 equiv, 2d6 vs chaotic

    // =====
    // WEAPON – Tactical / Combat Enhancement
    // =====
    Keen,              // +1 equiv, double threat range
    Speed,             // +3 equiv, extra attack at highest BAB
    Defending,         // +1 equiv, transfer enhancement to AC
    Dancing,           // +4 equiv, autonomous fighting 4 rounds
    Vorpal,            // +5 equiv, sever head on confirmed nat 20
    BrilliantEnergy,   // +4 equiv, ignore armor/shield/natural AC

    // =====
    // WEAPON – Situational / Creature-Type
    // =====
    Bane,              // +1 equiv, +2 atk/+2d6 vs creature type
    Disruption,        // +2 equiv, destroy undead DC 14 Will
    GhostTouchWeapon,  // +1 equiv, strike incorporeal
    Merciful,          // +1 equiv, nonlethal +1d6
    Thundering,        // +1 equiv, 1d8 sonic on crit + deafen

    // =====
    // WEAPON – Special Mechanics
    // =====
    SpellStoring,      // +1 equiv, store/discharge spell ≤ 3rd
    Wounding,          // +2 equiv, 1 Con damage per hit
    Vicious,           // +1 equiv, +2d6 dmg, 1d6 to self
    MightyCleaving,     // +1 equiv, +1 cleave/round
    KiFocus,           // +1 equiv, monk abilities through weapon
    Throwing,          // +1 equiv, melee becomes throwable

    // =====
    // WEAPON – Ranged-Only
    // =====
    Distance,          // +1 equiv, double range increment
    Seeking,           // +1 equiv, ignore concealment miss chance

```

```

Returning,          // +1 equiv, thrown weapon comes back

// =====
//  ARMOR/SHIELD – Fortification
// =====
FortificationLight,    // +1 equiv, 25% negate crit/sneak
FortificationModerate, // +3 equiv, 75% negate crit/sneak
FortificationHeavy,    // +5 equiv, 100% negate crit/sneak

// =====
//  ARMOR/SHIELD – Spell Resistance
// =====
SpellResistance13,    // +2 equiv
SpellResistance15,    // +3 equiv
SpellResistance17,    // +4 equiv
SpellResistance19,    // +5 equiv

// =====
//  ARMOR – Defensive
// =====
GhostTouchArmor,      // +3 equiv, enhancement vs incorporeal touch
Invulnerability,      // +3 equiv, DR 5/magic
Wild,                 // +3 equiv, AC in wild shape

// =====
//  ARMOR – Skill Bonuses (flat cost, base/improved/greater)
// =====
Shadow,               // 3750 gp, +5 Hide
ShadowImproved,       // 15000 gp, +10 Hide
ShadowGreater,        // 33750 gp, +15 Hide
SilentMoves,          // 3750 gp, +5 Move Silently
SilentMovesImproved,  // 15000 gp, +10 Move Silently
SilentMovesGreater,   // 33750 gp, +15 Move Silently
Slick,                // 3750 gp, +5 Escape Artist
SlickImproved,        // 15000 gp, +10 Escape Artist
SlickGreater,         // 33750 gp, +15 Escape Artist
Glamerd,              // 2700 gp, disguise as clothing

// =====
//  ARMOR/SHIELD – Energy Resistance (flat cost)
// =====
AcidResistance,        // 18000 gp, resist 10
AcidResistanceImproved, // 42000 gp, resist 20
AcidResistanceGreater, // 66000 gp, resist 30
ColdResistance,        // 18000 gp, resist 10
ColdResistanceImproved, // 42000 gp, resist 20
ColdResistanceGreater, // 66000 gp, resist 30
ElectricityResistance, // 18000 gp, resist 10
ElectricityResistanceImproved, // 42000 gp, resist 20
ElectricityResistanceGreater, // 66000 gp, resist 30
FireResistance,        // 18000 gp, resist 10
FireResistanceImproved, // 42000 gp, resist 20
FireResistanceGreater, // 66000 gp, resist 30
SonicResistance,       // 18000 gp, resist 10
SonicResistanceImproved, // 42000 gp, resist 20
SonicResistanceGreater, // 66000 gp, resist 30

// =====
//  ARMOR/SHIELD – Special (flat cost)
// =====
Etherealness,         // 49000 gp, 1/day ethereal jaunt
UndeadControlling,    // 49000 gp, control 26 HD undead/day

```

```
// =====  
//  SHIELD – Specific  
// =====  
ArrowCatching,    // +1 equiv, +1 AC vs ranged, attract for allies  
Bashing,          // +1 equiv, larger bash damage, enh applies  
Blinding,         // +1 equiv, 2/day blind flash DC 14  
Animated,         // +2 equiv, shield floats, frees hands  
ArrowDeflection,  // +2 equiv, 1/round DC 20 Reflex deflect  
Reflecting,       // +5 equiv, 1/day reflect spell  
}
```

3b. EnchantmentData Class

```

/// <summary>
/// Immutable data record for a single enchantment special ability.
/// Populated by EnchantmentProperties and attached to items.
/// </summary>
[System.Serializable]
public class EnchantmentData
{
    // — Identity —
    public EnchantmentType Type;
    public string Name;           // Display name: "Flaming", "Holy", etc.
    public string Description;    // Tooltip description

    // — Cost —
    public int BonusEquivalent;   // 0-5. If 0, uses FlatCostGp instead.
    public int FlatCostGp;       // Flat GP cost (for non-bonus abilities like
energy resist)

    // — Applicability —
    public bool CanApplyToWeapon;
    public bool CanApplyToArmor;
    public bool CanApplyToShield;
    public bool MeleeOnly;       // Defending, Dancing, Vorpall, etc.
    public bool RangedOnly;      // Distance, Seeking, Returning
    public bool SlashingOnly;    // Vorpall
    public bool SlashingOrPiercingOnly; // Keen
    public bool BludgeoningOnly; // Disruption
    public bool ThrownOnly;      // Returning

    // — Prerequisites —
    public EnchantmentType[] RequiredEnchantments; // e.g., Vorpall might require
nothing per RAW
    public int MinimumEnhancementBonus;           // Usually 1 (all require +1 base)
    public int CasterLevel;                       // Required CL for creation

    // — Weapon: Extra Damage on Hit —
    public int ExtraDamageDice;                   // Number of dice (1 for 1d6, 2 for 2d6)
    public int ExtraDamageDieSize;                // Die size (6 for d6, 8 for d8)
    public DamageType ExtraDamageType;            // Fire, Cold, Electricity, Sonic, Holy, etc.
    public bool ExtraDamageIsNonlethal;           // Merciful

    // — Weapon: Crit Bonus Damage —
    public int CritBonusDice;                     // Number of dice on crit (1 for 1d10)
    public int CritBonusDieSize;                  // Die size on crit (10 for d10)
    public int CritBonusPerMultiplier;           // Extra dice per crit multiplier above x2

    // — Weapon: Attack/Damage Modifiers —
    public int BaneAttackBonus;                   // +2 for Bane vs designated type
    public int BaneDamageDice;                   // 2 for 2d6 Bane damage
    public int BaneDamageDieSize;                // 6 for d6
    public bool DoubleThreatRange;               // Keen
    public bool GrantsExtraAttack;               // Speed
    public int SelfDamageDice;                   // Vicious: 1d6 to wielder
    public int SelfDamageDieSize;                // 6
    public int ConDamagePerHit;                  // Wounding: 1

    // — Weapon: Special Flags —
    public bool IgnoresArmorShieldNatural;       // Brilliant Energy
    public bool StrikesIncorporeal;              // Ghost Touch
    public bool DestroyUndeadOnHit;              // Disruption
    public int DestroyUndeadDC;                  // 14 for Disruption
    public bool SeverHeadOnNat20;               // Vorpall
    public bool DancesAutonomously;             // Dancing

```



```

    public int DancingRounds;           // 4
    public bool TransferEnhancementToAC; // Defending
    public bool CanStoreSpell;          // Spell Storing
    public int MaxStoredSpellLevel;     // 3
    public int ExtraCleavePerRound;     // Mighty Cleaving: 1
    public bool MonkKiChannel;          // Ki Focus
    public bool MakesThrowable;       // Throwing
    public int ThrowRangeIncrement;     // 10 ft

    // — Weapon: Ranged Specific —
    public bool DoubleRangeIncrement;    // Distance
    public bool IgnoresConcealment;     // Seeking
    public bool ReturnsAfterThrow;      // Returning

    // — Weapon: Alignment —
    public Alignment RequiredAlignment;  // For wielding without neg level
    public Alignment TargetAlignment;    // Which alignment takes extra damage
    public bool NegLevelOnWrongAlignment; // Holy/Unholy/Anarchic/Axiomatic

    // — Armor/Shield: Fortification —
    public int FortificationPercent;     // 25, 75, or 100

    // — Armor/Shield: Spell Resistance —
    public int GrantedSpellResistance;   // 13, 15, 17, or 19

    // — Armor/Shield: Energy Resistance —
    public DamageType ResistEnergyType;  // Which energy type
    public int EnergyResistanceAmount;    // 10, 20, or 30

    // — Armor: Damage Reduction —
    public int DRAmount;                 // 5 for Invulnerability
    public DamageBypassTag DRBypass;     // Magic for Invulnerability

    // — Armor: Skill Bonuses —
    public string SkillBonusName;        // "Hide", "Move Silently", "Escape
Artist"
    public int SkillBonusAmount;         // 5, 10, or 15

    // — Shield: Specific —
    public bool AnimatesShield;          // Animated
    public bool DeflectsArrows;          // Arrow Deflection
    public int ArrowDeflectionDC;        // 20
    public bool CatchesArrowsForAllies;  // Arrow Catching
    public int ArrowCatchingACBonus;     // +1 vs ranged
    public bool BashingShield;           // Bashing
    public bool BlindingFlash;           // Blinding
    public int BlindingDC;               // 14
    public int BlindingUsesPerDay;       // 2
    public bool ReflectsSpells;          // Reflecting
    public int ReflectUsesPerDay;        // 1

    // — Armor: Special —
    public bool WildShapeCompatible;     // Wild
    public bool EtherealOncePerDay;     // Etherealness
    public bool ControlsUndead;         // Undead Controlling
    public int UndeadControlHD;         // 26
    public bool GhostTouchAC;           // Ghost Touch armor
    public bool DisguisesAsClothing;    // Glamered

    // — Bane: Creature Type —
    public CreatureType BaneCreatureType; // For Bane weapons
}

```

3c. Alignment and DamageType Enums (extend existing or new)

```
/// <summary>
/// Alignment axis values for enchantment alignment checks.
/// </summary>
public enum Alignment
{
    None,
    Good, Evil,
    Lawful, Chaotic,
    // Combined:
    LawfulGood, NeutralGood, ChaoticGood,
    LawfulNeutral, TrueNeutral, ChaoticNeutral,
    LawfulEvil, NeutralEvil, ChaoticEvil
}

/// <summary>
/// Damage types used by enchantments and spells.
/// Extend existing DamageType enum if it exists.
/// </summary>
public enum DamageType
{
    None,
    // Physical
    Slashing, Piercing, Bludgeoning,
    // Energy
    Fire, Cold, Electricity, Sonic, Acid,
    // Alignment (for holy/unholy damage)
    Holy, Unholy, Anarchic, Axiomatic,
    // Other
    Force, Positive, Negative, Nonlethal
}
```

4. EnchantmentProperties — Central Database

```

/// <summary>
/// Single source of truth for all enchantment stats.
/// Mirror of MaterialProperties.cs architecture.
/// No enchantment data should be defined outside this file.
/// </summary>
public static class EnchantmentProperties
{
    private static readonly Dictionary<EnchantmentType, EnchantmentData> _database
        = new Dictionary<EnchantmentType, EnchantmentData>();

    static EnchantmentProperties()
    {
        InitializeDatabase();
    }

    /// <summary>
    /// Retrieve the data for any enchantment type.
    /// </summary>
    public static EnchantmentData Get(EnchantmentType type)
    {
        return _database.TryGetValue(type, out var data) ? data : null;
    }

    /// <summary>
    /// Get all enchantments valid for a given item.
    /// </summary>
    public static List<EnchantmentData> GetValidForItem(ItemData item)
    {
        var valid = new List<EnchantmentData>();
        foreach (var kvp in _database)
        {
            if (EnchantmentValidator.CanApplyEnchantment(item, kvp.Key))
                valid.Add(kvp.Value);
        }
        return valid;
    }

    /// <summary>
    /// Display name for building enchanted item names.
    /// Returns enchantment prefixes in canonical order.
    /// </summary>
    public static string GetEnchantmentPrefix(List<EnchantmentType> enchantments)
    {
        // Sort by display priority: alignment > elemental > tactical > other
        // E.g., "+1 Holy Flaming Burst Keen Longsword"
        var sorted = new List<EnchantmentType>(enchantments);
        sorted.Sort((a, b) => GetDisplayOrder(a).CompareTo(GetDisplayOrder(b)));

        var prefixes = new List<string>();
        foreach (var ench in sorted)
        {
            var data = Get(ench);
            if (data != null && !string.IsNullOrEmpty(data.Name))
                prefixes.Add(data.Name);
        }
        return string.Join(" ", prefixes);
    }

    private static void InitializeDatabase()
    {
        // — WEAPON: Elemental +1d6 —
        Register(new EnchantmentData

```

```

    {
        Type = EnchantmentType.Flaming,
        Name = "Flaming",
        Description = "Wreathed in fire, dealing 1d6 extra fire damage on hit.",
        BonusEquivalent = 1,
        CanApplyToWeapon = true,
        CasterLevel = 10,
        ExtraDamageDice = 1, ExtraDamageDieSize = 6,
        ExtraDamageType = DamageType.Fire
    });

    Register(new EnchantmentData
    {
        Type = EnchantmentType.Frost,
        Name = "Frost",
        Description = "Sheathed in ice, dealing 1d6 extra cold damage on hit.",
        BonusEquivalent = 1,
        CanApplyToWeapon = true,
        CasterLevel = 8,
        ExtraDamageDice = 1, ExtraDamageDieSize = 6,
        ExtraDamageType = DamageType.Cold
    });

    // ... (all 70+ enchantments defined here in the same pattern)
    // Each enchantment is a single Register() call with all fields set.
    // This is the ONLY place enchantment data lives.
}

private static void Register(EnchantmentData data)
{
    _database[data.Type] = data;
}

private static int GetDisplayOrder(EnchantmentType type)
{
    // Alignment first, then elemental, then tactical, etc.
    // Returns a sort key for building display names.
    // Implementation omitted for brevity.
    return (int)type;
}
}

```

5. ItemEnchantment — Component Model

Changes to ItemData.cs

```

// ADD to ItemData class:

/// <summary>
/// List of special abilities on this item. Empty for non-magical items.
/// All bonuses computed at runtime from this list + EnhancementBonus.
/// </summary>
public List<EnchantmentType> SpecialAbilities = new List<EnchantmentType>();

/// <summary>
/// For Bane weapons: which creature type the Bane targets.
/// Only relevant if SpecialAbilities contains EnchantmentType.Bane.
/// </summary>
public CreatureType BaneTargetType;

/// <summary>
/// For Spell Storing: the currently stored spell (null if empty).
/// </summary>
public SpellData StoredSpell;

/// <summary>
/// Total bonus equivalent = EnhancementBonus + sum of all special ability equivalent-
ents.
/// Used for pricing and the +10 cap validation.
/// </summary>
public int TotalBonusEquivalent
{
    get
    {
        int total = Mathf.Max(0, ResolveEnhancementBonus());
        foreach (var ability in SpecialAbilities)
        {
            var data = EnchantmentProperties.Get(ability);
            if (data != null)
                total += data.BonusEquivalent;
        }
        return total;
    }
}

/// <summary>
/// Total flat cost from enchantments that use GP pricing instead of bonus equiv.
/// </summary>
public int TotalFlatEnchantmentCostGp
{
    get
    {
        int total = 0;
        foreach (var ability in SpecialAbilities)
        {
            var data = EnchantmentProperties.Get(ability);
            if (data != null && data.BonusEquivalent == 0)
                total += data.FlatCostGp;
        }
        return total;
    }
}

/// <summary>
/// Check if this item has a specific enchantment.
/// </summary>
public bool HasEnchantment(EnchantmentType type)
{

```

```

        return SpecialAbilities.Contains(type);
    }

    /// <summary>
    /// Get fortification percentage from all sources (armor + shield).
    /// Returns highest fortification on the item.
    /// </summary>
    public int FortificationPercent
    {
        get
        {
            int best = 0;
            foreach (var ability in SpecialAbilities)
            {
                var data = EnchantmentProperties.Get(ability);
                if (data != null && data.FortificationPercent > best)
                    best = data.FortificationPercent;
            }
            return best;
        }
    }

    /// <summary>
    /// Get granted spell resistance from this item.
    /// </summary>
    public int GrantedSpellResistance
    {
        get
        {
            int best = 0;
            foreach (var ability in SpecialAbilities)
            {
                var data = EnchantmentProperties.Get(ability);
                if (data != null && data.GrantedSpellResistance > best)
                    best = data.GrantedSpellResistance;
            }
            return best;
        }
    }
}

```


Update to FullDisplayName

```
// MODIFY FullDisplayName property:
public string FullDisplayName
{
    get
    {
        string baseName = Name;
        int enhBonus = ResolveEnhancementBonus();

        // Material prefix
        string matPrefix = "";
        if (Material != null && Material.MaterialType != ItemMaterialType.Standard)
            matPrefix = MaterialProperties.GetMaterialPrefix(Material.MaterialType);

        // Enchantment prefix (before enhancement number)
        string enchPrefix = "";
        if (SpecialAbilities != null && SpecialAbilities.Count > 0)
            enchPrefix = EnchantmentProperties.GetEnchantmentPrefix(SpecialAbilities);

        // Enhancement prefix
        string enhPrefix = enhBonus > 0 ? $"#{enhBonus}" : "";

        // Masterwork prefix (only if no enhancement and no enchantments)
        string mwPrefix = "";
        if (IsMasterwork && enhBonus <= 0 && (SpecialAbilities == null || SpecialAbilities.Count == 0))
            mwPrefix = "Masterwork";

        // Build: "+2 Holy Flaming Adamantine Longsword"
        // Order: enhancement > enchantments > material > base name
        var parts = new List<string>();
        if (!string.IsNullOrEmpty(enhPrefix)) parts.Add(enhPrefix);
        if (!string.IsNullOrEmpty(enchPrefix)) parts.Add(enchPrefix);
        if (!string.IsNullOrEmpty(matPrefix)) parts.Add(matPrefix);
        if (!string.IsNullOrEmpty(mwPrefix)) parts.Add(mwPrefix);
        parts.Add(baseName);

        return string.Join(" ", parts);
    }
}
```

6. EnchantmentFactory — Variant Generation

```

public static class EnchantmentFactory
{
    /// <summary>
    /// Create an enchanted version of a base item.
    /// Item must already be masterwork. Enhancement bonus must be ≥ 1.
    /// </summary>
    public static ItemData CreateEnchantedItem(
        ItemData baseItem,
        int enhancementBonus,
        params EnchantmentType[] specialAbilities)
    {
        if (baseItem == null) return null;
        if (enhancementBonus < 1 || enhancementBonus > 5) return null;

        // Validate all enchantments
        foreach (var ability in specialAbilities)
        {
            if (!EnchantmentValidator.CanApplyEnchantmentType(baseItem, ability))
                return null;
        }

        // Validate total doesn't exceed +10
        int totalEquiv = enhancementBonus;
        foreach (var ability in specialAbilities)
        {
            var data = EnchantmentProperties.Get(ability);
            if (data != null) totalEquiv += data.BonusEquivalent;
        }
        if (totalEquiv > 10) return null;

        // Clone and configure
        var clone = ItemDatabase.CloneItem(baseItem.Id);
        if (clone == null) return null;

        clone.IsMasterwork = true;
        clone.EnhancementBonus = enhancementBonus;
        clone.SpecialAbilities = new List<EnchantmentType>(specialAbilities);

        // Calculate price
        clone.BasePriceGp = EnchantmentPricing.CalculateTotalPrice(
            baseItem.BasePriceGp, baseItem, enhancementBonus, specialAbilities);

        // Generate ID and leave Name for FullDisplayName to handle
        string enchId = BuildEnchantmentId(enhancementBonus, specialAbilities);
        clone.Id = $"{enchId}_{baseItem.Id}";

        return clone;
    }

    /// <summary>
    /// Apply an enchantment to an existing item in-place.
    /// Returns true if successful.
    /// </summary>
    public static bool ApplyEnchantment(ItemData item, EnchantmentType enchantment)
    {
        if (item == null) return false;
        if (!EnchantmentValidator.CanApplyEnchantment(item, enchantment)) return
false;

        item.SpecialAbilities.Add(enchantment);

        // Recalculate price
    }
}

```

```

        item.BasePriceGp = EnchantmentPricing.CalculateTotalPrice(
            item.BasePriceGp, item, item.ResolveEnhancementBonus(),
            item.SpecialAbilities.ToArray());

        return true;
    }

    /// <summary>
    /// Register common enchanted weapon/armor variants at initialization.
    /// Called after ItemMaterialFactory.RegisterAllMaterialVariants().
    /// </summary>
    public static void RegisterCommonVariants()
    {
        int count = 0;

        // +1 through +5 versions of common weapons
        foreach (string weaponId in CommonWeaponIds)
        {
            for (int enh = 1; enh <= 5; enh++)
            {
                RegisterVariant(CreateEnchantedItem(
                    ItemDatabase.Get(weaponId), enh), ref count);
            }

            // Common +1 with special ability combos
            foreach (var ability in Tier1WeaponAbilities)
            {
                RegisterVariant(CreateEnchantedItem(
                    ItemDatabase.Get(weaponId), 1, ability), ref count);
            }
        }

        // +1 through +5 versions of common armor
        foreach (string armorId in CommonArmorIds)
        {
            for (int enh = 1; enh <= 5; enh++)
            {
                RegisterVariant(CreateEnchantedItem(
                    ItemDatabase.Get(armorId), enh), ref count);
            }
        }

        Debug.Log($"[EnchantmentFactory] Registered {count} enchanted variants.");
    }

    private static string BuildEnchantmentId(int enh, EnchantmentType[] abilities)
    {
        var parts = new List<string> { $"plus{enh}" };
        foreach (var a in abilities)
            parts.Add(a.ToString().ToLowerInvariant());
        return string.Join("_", parts);
    }
}

```

7. EnchantmentEffects — Runtime Combat Integration

```

/// <summary>
/// Applies enchantment effects during combat resolution.
/// Called from CharacterController.PerformSingleAttackWithCrit() and
/// Inventory.RecalculateStats().
/// </summary>
public static class EnchantmentEffects
{
    // =====
    // WEAPON EFFECTS – Attack Phase
    // =====

    /// <summary>
    /// Get total extra attack bonus from weapon enchantments.
    /// Called before the attack roll.
    /// </summary>
    public static int GetAttackModifier(ItemData weapon, CharacterController target)
    {
        int bonus = 0;
        if (weapon?.SpecialAbilities == null) return bonus;

        foreach (var ability in weapon.SpecialAbilities)
        {
            var data = EnchantmentProperties.Get(ability);
            if (data == null) continue;

            // Bane: +2 attack vs designated creature type
            if (ability == EnchantmentType.Bane &&
                target != null &&
                IsCreatureType(target, weapon.BaneTargetType))
            {
                bonus += data.BaneAttackBonus; // +2
            }
        }
        return bonus;
    }

    /// <summary>
    /// Check if weapon has Keen (for crit range doubling).
    /// Applied in threat range calculation.
    /// </summary>
    public static bool HasKeenEnchantment(ItemData weapon)
    {
        return weapon?.HasEnchantment(EnchantmentType.Keen) ?? false;
    }

    /// <summary>
    /// Check if weapon grants extra attack (Speed).
    /// </summary>
    public static bool GrantsExtraAttack(ItemData weapon)
    {
        return weapon?.HasEnchantment(EnchantmentType.Speed) ?? false;
    }

    // =====
    // WEAPON EFFECTS – Damage Phase
    // =====

    /// <summary>
    /// Roll all extra damage from weapon enchantments.
    /// Called after a successful hit, before DR.
    /// Returns a list of (amount, type) tuples.
    /// </summary>

```

```

    public static List<(int amount, DamageType type, string source)>
        RollExtraDamage(ItemData weapon, CharacterController target, bool isCrit, int
critMultiplier)
    {
        var damages = new List<(int, DamageType, string)>();
        if (weapon?.SpecialAbilities == null) return damages;

        foreach (var ability in weapon.SpecialAbilities)
        {
            var data = EnchantmentProperties.Get(ability);
            if (data == null) continue;

            // Standard extra damage on every hit (Flaming 1d6, etc.)
            if (data.ExtraDamageDice > 0)
            {
                int dmg = RollDice(data.ExtraDamageDice, data.ExtraDamageDieSize);
                damages.Add((dmg, data.ExtraDamageType, data.Name));
            }

            // Crit bonus damage (Burst abilities: 1d10 per crit mult above x1)
            if (isCrit && data.CritBonusDice > 0)
            {
                int extraDice = data.CritBonusDice;
                if (data.CritBonusPerMultiplier > 0)
                    extraDice = data.CritBonusPerMultiplier * (critMultiplier - 1);
                int critDmg = RollDice(extraDice, data.CritBonusDieSize);
                damages.Add((critDmg, data.ExtraDamageType, $"{data.Name} Burst"));
            }

            // Alignment damage (Holy 2d6 vs evil, etc.)
            if (data.TargetAlignment != Alignment.None && target != null)
            {
                if (MatchesAlignment(target, data.TargetAlignment))
                {
                    int alignDmg = RollDice(data.ExtraDamageDice, data.ExtraDamageDieS
ize);
                    damages.Add((alignDmg, data.ExtraDamageType, data.Name));
                }
            }

            // Bane extra damage
            if (ability == EnchantmentType.Bane &&
                target != null &&
                IsCreatureType(target, weapon.BaneTargetType))
            {
                int baneDmg = RollDice(data.BaneDamageDice, data.BaneDamageDieSize);
                damages.Add((baneDmg, DamageType.None, "Bane"));
            }

            // Thundering on crit
            if (ability == EnchantmentType.Thundering && isCrit)
            {
                int sonicDmg = RollDice(1, 8);
                damages.Add((sonicDmg, DamageType.Sonic, "Thundering"));
                // DC 14 Fort or deafened – handled in special effects
            }

            // Vicious: self-damage
            if (ability == EnchantmentType.Vicious)
            {
                int selfDmg = RollDice(data.SelfDamageDice, data.SelfDamageDieSize);
                // Apply self-damage to wielder (handled by caller)
            }
        }
    }

```

```

    }

    return damages;
}

// =====
// WEAPON EFFECTS – Special Triggers
// =====

/// <summary>
/// Check and apply Disruption (destroy undead on hit).
/// </summary>
public static bool CheckDisruption(ItemData weapon, CharacterController target)
{
    if (!weapon.HasEnchantment(EnchantmentType.Disruption)) return false;
    if (!target.Stats.IsUndead) return false;

    int dc = 14;
    int save = target.RollWillSave();
    return save < dc; // true = undead destroyed
}

/// <summary>
/// Check Vorpal effect on confirmed nat 20 crit.
/// </summary>
public static bool CheckVorpal(ItemData weapon, int naturalRoll, bool critCon-
firmed)
{
    if (!weapon.HasEnchantment(EnchantmentType.Vorpal)) return false;
    if (naturalRoll != 20 || !critConfirmed) return false;
    return true; // Caller handles head severing
}

/// <summary>
/// Apply Wounding (Constitution damage).
/// </summary>
public static void ApplyWounding(ItemData weapon, CharacterController target)
{
    if (!weapon.HasEnchantment(EnchantmentType.Wounding)) return;
    target.Stats.ApplyAbilityDamage("Constitution", 1);
}

// =====
// ARMOR/SHIELD EFFECTS – Defense Phase
// =====

/// <summary>
/// Check fortification: should this crit/sneak be negated?
/// Returns true if the crit/sneak is negated (becomes normal hit).
/// </summary>
public static bool CheckFortification(CharacterController defender)
{
    int percent = 0;

    // Check armor
    var armor = defender.Inventory?.ArmorRobeSlot;
    if (armor != null) percent = Mathf.Max(percent, armor.FortificationPercent);

    // Check shield
    var shield = defender.Inventory?.LeftHandSlot;
    if (shield != null && shield.IsShield)
        percent = Mathf.Max(percent, shield.FortificationPercent);
}

```



```

        if (percent <= 0) return false;

        int roll = Random.Range(1, 101); // 1-100
        return roll <= percent; // true = negated
    }

    /// <summary>
    /// Get total energy resistance from armor + shield enchantments.
    /// Returns resistance amount for the given damage type.
    /// </summary>
    public static int GetEnergyResistance(CharacterController defender, DamageType type)
    {
        int total = 0;

        var armor = defender.Inventory?.ArmorRobeSlot;
        if (armor != null)
            total += GetItemEnergyResistance(armor, type);

        var shield = defender.Inventory?.LeftHandSlot;
        if (shield != null && shield.IsShield)
            total += GetItemEnergyResistance(shield, type);

        return total;
    }

    /// <summary>
    /// Get spell resistance granted by equipment.
    /// Uses highest (does not stack).
    /// </summary>
    public static int GetEquipmentSpellResistance(CharacterController character)
    {
        int best = 0;

        var armor = character.Inventory?.ArmorRobeSlot;
        if (armor != null) best = Mathf.Max(best, armor.GrantedSpellResistance);

        var shield = character.Inventory?.LeftHandSlot;
        if (shield != null && shield.IsShield)
            best = Mathf.Max(best, shield.GrantedSpellResistance);

        return best;
    }

    // =====
    // SHIELD-SPECIFIC EFFECTS
    // =====

    /// <summary>
    /// Check Arrow Deflection: DC 20 Reflex to negate ranged hit.
    /// </summary>
    public static bool CheckArrowDeflection(CharacterController defender)
    {
        var shield = defender.Inventory?.LeftHandSlot;
        if (shield == null || !shield.HasEnchantment(EnchantmentType.ArrowDeflection))
            return false;
        if (defender.IsFlatFooted) return false;

        int reflexSave = defender.RollReflexSave();
        return reflexSave >= 20;
    }

    // — Helper Methods —

```

```
private static int RollDice(int count, int sides)
{
    int total = 0;
    for (int i = 0; i < count; i++)
        total += Random.Range(1, sides + 1);
    return total;
}

private static int GetItemEnergyResistance(ItemData item, DamageType type)
{
    int total = 0;
    if (item?.SpecialAbilities == null) return total;

    foreach (var ability in item.SpecialAbilities)
    {
        var data = EnchantmentProperties.Get(ability);
        if (data != null && data.ResistEnergyType == type)
            total += data.EnergyResistanceAmount;
    }
    return total;
}

private static bool IsCreatureType(CharacterController target, CreatureType type)
{
    // Check target's creature type tag against Bane designation
    return target.Stats.CreatureType == type;
}

private static bool MatchesAlignment(CharacterController target, Alignment target-
Alignment)
{
    // Check if target's alignment matches for Holy/Unholy/etc.
    return target.Stats.HasAlignment(targetAlignment);
}
}
```

8. Price Calculation System

```

public static class EnchantmentPricing
{
    // =====
    // DMG Price Formulas (PHB p.126, DMG p.217, 222)
    // =====
    //
    // Weapons: (total bonus equiv)2 × 2,000 gp + MW cost (300) + base weapon cost
    // Armor: (total bonus equiv)2 × 1,000 gp + MW cost (150) + base armor cost
    // Shields: (total bonus equiv)2 × 1,000 gp + MW cost (150) + base shield cost
    //
    // Flat-cost abilities are added on top (not in the bonus-squared formula).
    //
    // Examples:
    // +1 Flaming Longsword: total equiv = +2
    // = 22 × 2000 + 300 + 15 = 8,315 gp
    //
    // +2 Full Plate of Moderate Fortification: total equiv = +5
    // = 52 × 1000 + 150 + 1500 = 26,650 gp

    /// <summary>
    /// Calculate total market price for an enchanted item.
    /// </summary>
    public static int CalculateTotalPrice(
        int baseItemCost,
        ItemData item,
        int enhancementBonus,
        EnchantmentType[] specialAbilities)
    {
        // 1. Calculate total bonus equivalent
        int totalEquiv = enhancementBonus;
        int flatCost = 0;

        foreach (var ability in specialAbilities)
        {
            var data = EnchantmentProperties.Get(ability);
            if (data == null) continue;

            if (data.BonusEquivalent > 0)
                totalEquiv += data.BonusEquivalent;
            else
                flatCost += data.FlatCostGp;
        }

        // 2. Bonus-squared multiplier
        int multiplier = item.IsWeapon ? 2000 : 1000; // Weapons vs armor/shields
        int bonusCost = totalEquiv * totalEquiv * multiplier;

        // 3. Masterwork cost
        int mwCost = MaterialProperties.GetMasterworkCost(item);

        // 4. Material cost (if any)
        int materialCost = item.Material?.AdditionalCostGp ?? 0;

        // 5. Total = base + MW + material + bonus-squared + flat
        return baseItemCost + mwCost + materialCost + bonusCost + flatCost;
    }

    /// <summary>
    /// Get the price table for reference.
    /// </summary>
    public static string GetPriceTable()
    {

```

```
return @"
Enhancement Equiv | Weapon Cost | Armor/Shield Cost
+1                | 2,000 gp   | 1,000 gp
+2                | 8,000 gp   | 4,000 gp
+3                | 18,000 gp  | 9,000 gp
+4                | 32,000 gp  | 16,000 gp
+5                | 50,000 gp  | 25,000 gp
+6                | 72,000 gp  | 36,000 gp
+7                | 98,000 gp  | 49,000 gp
+8                | 128,000 gp | 64,000 gp
+9                | 162,000 gp | 81,000 gp
+10               | 200,000 gp | 100,000 gp
(+ masterwork + base item + material + flat-cost abilities)";
}
```

9. Validation System

```

public static class EnchantmentValidator
{
    /// <summary>
    /// Check if a specific enchantment can be added to an item.
    /// Validates all rules: type compatibility, restrictions, cap, duplicates.
    /// </summary>
    public static bool CanApplyEnchantment(ItemData item, EnchantmentType enchantment)
    {
        if (item == null) return false;

        var data = EnchantmentProperties.Get(enchantment);
        if (data == null) return false;

        // Must be masterwork
        if (!item.IsMasterwork) return false;

        // Must have at least +1 enhancement
        if (item.ResolveEnhancementBonus() < 1) return false;

        // Check item type compatibility
        if (item.IsWeapon && !data.CanApplyToWeapon) return false;
        if (item.IsArmor && !data.CanApplyToArmor) return false;
        if (item.IsShield && !data.CanApplyToShield) return false;

        // Check melee/ranged restrictions
        if (data.MeleeOnly && item.IsRangedWeapon) return false;
        if (data.RangedOnly && !item.IsRangedWeapon) return false;

        // Check weapon type restrictions
        if (data.SlashingOnly && item.DamageTypeTag != "Slashing") return false;
        if (data.SlashingOrPiercingOnly &&
            item.DamageTypeTag != "Slashing" && item.DamageTypeTag != "Piercing") re-
turn false;
        if (data.BludgeoningOnly && item.DamageTypeTag != "Bludgeoning") return false;
        if (data.ThrownOnly && !item.IsThrown) return false;

        // Check no duplicates
        if (item.SpecialAbilities.Contains(enchantment)) return false;

        // Check mutually exclusive enchantments
        if (!CheckExclusivity(item, enchantment)) return false;

        // Check total bonus equivalent cap
        int newTotal = item.TotalBonusEquivalent + data.BonusEquivalent;
        if (newTotal > 10) return false;

        // Check prerequisites
        if (data.RequiredEnchantments != null)
        {
            foreach (var req in data.RequiredEnchantments)
            {
                if (!item.SpecialAbilities.Contains(req)) return false;
            }
        }

        return true;
    }

    /// <summary>
    /// Simple type-only check (doesn't require item instance).
    /// </summary>
    public static bool CanApplyEnchantmentType(ItemData item, EnchantmentType enchant-

```

```

ment)
{
    var data = EnchantmentProperties.Get(enchantment);
    if (data == null) return false;
    if (item.IsWeapon && !data.CanApplyToWeapon) return false;
    if (item.IsArmor && !data.CanApplyToArmor) return false;
    if (item.IsShield && !data.CanApplyToShield) return false;
    return true;
}

/// <summary>
/// Check for mutually exclusive enchantments.
/// E.g., Flaming and Flaming Burst are technically compatible (burst includes
flaming),
/// but Light/Moderate/Heavy Fortification are mutually exclusive.
/// </summary>
private static bool CheckExclusivity(ItemData item, EnchantmentType newEnchant-
ment)
{
    // Fortification: only one level
    if (IsFortification(newEnchantment))
    {
        foreach (var existing in item.SpecialAbilities)
            if (IsFortification(existing)) return false;
    }

    // Spell Resistance: only one level
    if (IsSpellResistance(newEnchantment))
    {
        foreach (var existing in item.SpecialAbilities)
            if (IsSpellResistance(existing)) return false;
    }

    // Energy Resistance: same type can't have multiple tiers
    // (e.g., Fire Resistance + Fire Resistance Improved = invalid)
    var newData = EnchantmentProperties.Get(newEnchantment);
    if (newData?.ResistEnergyType != DamageType.None)
    {
        foreach (var existing in item.SpecialAbilities)
        {
            var existingData = EnchantmentProperties.Get(existing);
            if (existingData?.ResistEnergyType == newData.ResistEnergyType)
                return false;
        }
    }

    // Burst supersedes base: Flaming Burst replaces Flaming
    // (Actually per SRD, Burst *includes* base, so having both is redundant but
not invalid)
    // We allow it but could warn.

    return true;
}

private static bool IsFortification(EnchantmentType t)
{
    return t == EnchantmentType.FortificationLight ||
           t == EnchantmentType.FortificationModerate ||
           t == EnchantmentType.FortificationHeavy;
}

private static bool IsSpellResistance(EnchantmentType t)
{

```



```
        return t == EnchantmentType.SpellResistance13 ||  
               t == EnchantmentType.SpellResistance15 ||  
               t == EnchantmentType.SpellResistance17 ||  
               t == EnchantmentType.SpellResistance19;  
    }  
}
```

10. Loot Generation

```

public static class EnchantmentLootGenerator
{
    /// <summary>
    /// Generate an appropriately enchanted weapon for the given CR.
    /// Based on DMG Table 7-14: Random Magic Weapon Generation.
    /// </summary>
    public static ItemData GenerateRandomMagicWeapon(string baseWeaponId, int cr)
    {
        ItemData baseWeapon = ItemDatabase.Get(baseWeaponId);
        if (baseWeapon == null) return null;

        // Determine enhancement bonus based on CR
        int enhancement = GetEnhancementForCR(cr);
        if (enhancement <= 0) return null;

        // Determine number and type of special abilities
        var abilities = new List<EnchantmentType>();
        int remainingBudget = 10 - enhancement;

        if (cr >= 7 && Random.value < 0.30f && remainingBudget >= 1)
        {
            var ability = SelectRandomWeaponAbility(baseWeapon, 1);
            if (ability.HasValue)
            {
                abilities.Add(ability.Value);
                remainingBudget -= EnchantmentProperties.Get(ability.Value).BonusEquivalent;
            }
        }

        if (cr >= 11 && Random.value < 0.20f && remainingBudget >= 2)
        {
            var ability = SelectRandomWeaponAbility(baseWeapon, 2);
            if (ability.HasValue)
            {
                abilities.Add(ability.Value);
            }
        }

        return EnchantmentFactory.CreateEnchantedItem(
            baseWeapon, enhancement, abilities.ToArray());
    }

    /// <summary>
    /// CR-based enhancement bonus distribution.
    /// </summary>
    private static int GetEnhancementForCR(int cr)
    {
        // CR 1-3: 10% +1
        // CR 4-6: 50% +1, 10% +2
        // CR 7-10: 30% +1, 30% +2, 10% +3
        // CR 11-15: 10% +1, 20% +2, 30% +3, 15% +4
        // CR 16-20: 5% +2, 15% +3, 30% +4, 20% +5

        float roll = Random.value;

        if (cr >= 16)
        {
            if (roll < 0.05f) return 2;
            if (roll < 0.20f) return 3;
            if (roll < 0.50f) return 4;
            if (roll < 0.70f) return 5;
            return 0;
        }
    }
}

```

```

    if (cr >= 11)
    {
        if (roll < 0.10f) return 1;
        if (roll < 0.30f) return 2;
        if (roll < 0.60f) return 3;
        if (roll < 0.75f) return 4;
        return 0;
    }
    if (cr >= 7)
    {
        if (roll < 0.30f) return 1;
        if (roll < 0.60f) return 2;
        if (roll < 0.70f) return 3;
        return 0;
    }
    if (cr >= 4)
    {
        if (roll < 0.50f) return 1;
        if (roll < 0.60f) return 2;
        return 0;
    }
    if (cr >= 1)
    {
        if (roll < 0.10f) return 1;
        return 0;
    }

    return 0;
}

private static EnchantmentType? SelectRandomWeaponAbility(
    ItemData weapon, int maxBonusEquiv)
{
    // Build list of valid abilities at this budget
    var candidates = new List<EnchantmentType>();
    foreach (EnchantmentType type in System.Enum.GetValues(typeof(Enchantment-
Type)))
    {
        var data = EnchantmentProperties.Get(type);
        if (data == null) continue;
        if (!data.CanApplyToWeapon) continue;
        if (data.BonusEquivalent > maxBonusEquiv) continue;
        if (data.BonusEquivalent <= 0) continue; // Skip flat-cost
        if (data.MeleeOnly && weapon.IsRangedWeapon) continue;
        if (data.RangedOnly && !weapon.IsRangedWeapon) continue;
        candidates.Add(type);
    }

    if (candidates.Count == 0) return null;
    return candidates[Random.Range(0, candidates.Count)];
}
}

```

11. Combat Integration Points

Where enchantment effects plug into existing code:

```

CharacterController.PerformSingleAttackWithCrit()
[ ]
[ ] BEFORE attack roll:
[ ] [ ] Check Keen [ ] double threat range
[ ] [ ] Check Speed [ ] grant extra attack in full attack
[ ] [ ] Check Defending [ ] transfer bonus to AC
[ ]
[ ] ATTACK ROLL:
[ ] [ ] Add EnchantmentEffects.GetAttackModifier(weapon, target) // Bane +2
[ ] [ ] existing masterworkAttackBonus + enhancementAttackBonus
[ ]
[ ] ON HIT:
[ ] [ ] Roll base weapon damage (existing)
[ ] [ ] Roll EnchantmentEffects.RollExtraDamage(weapon, target, isCrit, critMult)
[ ] [ ] [ ] Flaming/Frost/Shock: +1d6
[ ] [ ] [ ] Burst on crit: +1d10 per crit mult
[ ] [ ] [ ] Holy/Unholy: +2d6 vs alignment
[ ] [ ] [ ] Bane: +2d6 vs creature type
[ ] [ ] [ ] Vicious: +2d6 to target, 1d6 to self
[ ] [ ] [ ] Thundering on crit: +1d8 sonic
[ ] [ ] [ ] Check Disruption vs undead [ ] destroy on failed Will save
[ ] [ ] [ ] Check Wounding [ ] 1 Con damage
[ ] [ ] [ ] Check Vorpai on nat 20 crit [ ] sever head
[ ]
[ ] ON CRITICAL CONFIRMED:
[ ] [ ] Check EnchantmentEffects.CheckFortification(defender)
[ ] [ ] [ ] If true: convert to normal hit (no crit multiplier)
[ ] [ ] [ ] Burst damage already handled in RollExtraDamage
[ ]
[ ] DAMAGE APPLICATION:
[ ] [ ] Check EnchantmentEffects.GetEnergyResistance(defender, type)
[ ] [ ] [ ] Subtract resistance from each energy damage type
[ ] [ ] [ ] Check Brilliant Energy [ ] skip armor/shield/natural AC
[ ]
[ ] DEFENSE PHASE (on being attacked):
[ ] [ ] Fortification check (above)
[ ] [ ] Arrow Deflection [ ] DC 20 Reflex negates ranged
[ ] [ ] Arrow Catching [ ] redirect ranged to shield bearer
[ ] [ ] Spell Resistance from equipment

```

Specific File Modifications Required:

File	Modification
ItemData.cs	Add SpecialAbilities , BaneTargetType , StoredSpell , computed properties
ItemDatabase.cs	Clone SpecialAbilities list; call EnchantmentFactory.RegisterCommonVariants() at init
CharacterController.cs	Add enchantment attack/damage calls in PerformSingleAttackWithCrit
CombatResult.cs	Add fields for enchantment damage breakdown
Inventory.cs	Read GrantedSpellResistance , FortificationPercent in RecalculateStats
CharacterStats.cs	Add EquipmentSpellResistance field set by Inventory
InventoryUI.cs / PreCombatInventoryUI.cs	Display enchantment info in tooltips
GameManager.NPCSetup.cs	Use EnchantmentLootGenerator for NPC equipment

12. Existing Codebase Integration

Already Implemented (leverage these):

- **Enhancement bonus system:** ItemData.ResolveEnhancementBonus() , GetEnhancementAttackBonus() , GetEnhancementDamageBonus() — already handles +1 to +5 weapon/armor enhancement
- **Material system:** ItemMaterial , MaterialProperties , ItemMaterialFactory — provides the architectural template
- **Crit system:** Threat range, crit confirmation, multipliers already in combat code
- **DR system:** AddDamageReduction() , RemoveDamageReduction() on CharacterStats
- **Spell Resistance:** Already implemented for spells (extend to equipment SR)
- **Haste/Speed:** Extra attack logic already exists for haste spell — reuse for Speed enchantment
- **Quality color:** GetQualityColor() already returns gold for magic items
- **FullDisplayName:** Already handles enhancement prefix — extend for enchantment prefixes

New Systems Needed:

- **Energy damage types:** Currently damage is not typed (just physical). Need to add damage type tracking and energy resistance application.
- **Alignment system:** Need character alignment for Holy/Unholy/Anarchic/Axiomatic checks.
- **Creature type system:** Need creature type tags for Bane. (Partially exists via CharacterTags .)

- **Fortification check:** New check in crit confirmation pipeline.
- **Equipment SR:** Extend existing SR checks to include equipment-granted SR.

13. Implementation Schedule

Phase 1: Foundation (Days 1-3)

- [] Create `ItemEnchantment.cs` (enum + data class)
- [] Create `EnchantmentProperties.cs` (all 70+ enchantments registered)
- [] Create `EnchantmentValidator.cs`
- [] Create `EnchantmentPricing.cs`
- [] Add `SpecialAbilities` list to `ItemData.cs`
- [] Add computed properties (`TotalBonusEquivalent` , `FortificationPercent` , etc.)
- [] Update `FullDisplayName` for enchantment prefixes
- [] Update `ItemDatabase.cs` to clone `SpecialAbilities`
- [] Git commit: "Add enchantment data structures and central properties database"

Phase 2: Factory & Registration (Days 4-5)

- [] Create `EnchantmentFactory.cs`
- [] Register +1 through +5 of common weapons
- [] Register +1 through +5 of common armor
- [] Register common ability combos (+1 Flaming, +1 Frost, +1 Shock, etc.)
- [] Update `ItemDatabase.Init()` to call `EnchantmentFactory.RegisterCommonVariants()`
- [] Git commit: "Add enchantment factory and variant registration"

Phase 3: Core Weapon Enchantments (Days 6-8)

- [] Create `EnchantmentEffects.cs`
- [] Implement elemental damage (Flaming/Frost/Shock + Burst variants)
- [] Implement Keen (crit range doubling)
- [] Implement Speed (extra attack — reuse haste logic)
- [] Integrate into `CharacterController.PerformSingleAttackWithCrit()`
- [] Add enchantment damage to `CombatResult` display
- [] Git commit: "Implement core weapon enchantment effects in combat"

Phase 4: Advanced Weapon Enchantments (Days 9-11)

- [] Implement alignment damage (Holy/Unholy/Anarchic/Axiomatic)
- [] Implement Bane (creature type targeting)
- [] Implement Vicious, Wounding, Disruption
- [] Implement Defending (AC transfer)
- [] Implement Vorpal, Thundering
- [] Add energy damage type system
- [] Git commit: "Implement advanced weapon enchantments"

Phase 5: Armor & Shield Enchantments (Days 12-14)

- [] Implement Fortification (crit negation check)
- [] Implement Spell Resistance from equipment

- [] Implement Energy Resistance (armor/shield)
- [] Implement Invulnerability (DR 5/magic)
- [] Implement Ghost Touch armor
- [] Implement Shield-specific: Animated, Bashing, Arrow Deflection, Blinding
- [] Integrate into `Inventory.RecalculateStats()` and combat pipeline
- [] Git commit: "Implement armor and shield enchantment effects"

Phase 6: Loot & NPC Integration (Days 15-16)

- [] Create `EnchantmentLootGenerator.cs`
- [] Integrate into `GameManager.NPCSetup.cs` for CR-based equipment
- [] Update existing material loot helpers to also consider enchantments
- [] Git commit: "Add enchantment loot generation and NPC equipment"

Phase 7: UI & Polish (Days 17-18)

- [] Update tooltips to show enchantment details
- [] Add enchantment damage breakdown to combat log
- [] Update `GetQualityColor()` for enchantment-tier coloring
- [] Skill bonuses from Shadow/Silent Moves/Slick
- [] Git commit: "Polish enchantment UI display and combat logging"

Phase 8: Testing & Documentation (Days 19-20)

- [] Write test suite (see below)
 - [] Create `ENCHANTMENT_SYSTEM_ARCHITECTURE.md`
 - [] Final validation pass
 - [] Git commit: "Complete enchantment system testing and documentation"
-

14. Testing Plan

Unit Tests

```
// Test file: Tests/Equipment/EnchantmentSystemTests.cs

[Test] public void FlamingLongsword_Deals1d6FireDamage() { ... }
[Test] public void FlamingBurst_DealsExtraDiceOnCrit() { ... }
[Test] public void HolyWeapon_Deals2d6VsEvil_NoneVsGood() { ... }
[Test] public void Keen_DoublesTheatRange_19to20_Becomes_17to20() { ... }
[Test] public void Speed_GrantsExtraAttack_DoesNotStackWithHaste() { ... }
[Test] public void Bane_Plus2AttackPlus2d6_VsDesignatedType() { ... }
[Test] public void Vorpal_SeverHead_OnNat20CritConfirmed() { ... }
[Test] public void Disruption_DestroyUndead_DC14Will() { ... }
[Test] public void Vicious_2d6ToTarget_1d6ToSelf() { ... }
[Test] public void Wounding_1ConDamagePerHit() { ... }
[Test] public void Defending_TransferEnhancementToAC() { ... }

[Test] public void LightFortification_25PercentNegateCrit() { ... }
[Test] public void ModerateFortification_75PercentNegateCrit() { ... }
[Test] public void HeavyFortification_100PercentNegateCrit() { ... }
[Test] public void SpellResistance15_GrantsSR15() { ... }
[Test] public void Invulnerability_DR5Magic() { ... }
[Test] public void FireResistance_Absorbs10FireDamage() { ... }
[Test] public void EnergyResist_ArmorAndShield_Stack() { ... }

[Test] public void Animated_ShieldFloats_FreesHands() { ... }
[Test] public void Bashing_IncreasesShieldBashDamage() { ... }
[Test] public void ArrowDeflection_DC20ReflexNegatesRanged() { ... }

[Test] public void TotalBonusEquiv_CannotExceed10() { ... }
[Test] public void CannotAddDuplicateEnchantment() { ... }
[Test] public void KeenRequiresSlashingOrPiercing() { ... }
[Test] public void DisruptionRequiresBludgeoning() { ... }
[Test] public void VorpalRequiresSlashing() { ... }
[Test] public void FortificationLevels_MutuallyExclusive() { ... }

[Test] public void PriceCalculation_Plus1Flaming_Is8315gp() { ... }
[Test] public void PriceCalculation_Plus5Vorpal_Is200350gp() { ... }
[Test] public void PriceCalculation_FlatCostAbilities_AddOnTop() { ... }

[Test] public void DisplayName_Plus1FlamingKeenLongsword() { ... }
[Test] public void DisplayName_Plus2HolyAdamantineLongsword() { ... }
```

Integration Tests

- Create a +1 Flaming Longsword, equip it, attack target → verify 1d6 fire in combat log
- Create +1 Heavy Fortification Full Plate, score crit → verify crit always negated
- Create +1 Speed Longsword, full attack → verify extra attack (and doesn't stack with haste)
- Create +2 Holy Adamantine Longsword → verify material + enchantment + enhancement all work together
- NPC at CR 10 → verify loot generation produces appropriate enchanted gear

Regression Tests

- All existing material system tests still pass
- All existing combat tests still pass
- All existing inventory tests still pass

- Masterwork attack bonus still suppressed when enhancement $\geq +1$
- Material DR (adamantine) still works alongside enchantment DR (invulnerability)

Appendix A: Enchantment Count Summary

Category	Bonus-Equiv Count	Flat-Cost Count	Total
Weapon (Melee + Ranged)	30	0	30
Armor	10	27	37
Shield	15	16	31
Unique (deduplicated)	~45	~30	~75

Note: Many abilities are shared between armor and shields (Fortification, SR, Energy Resistance, Ghost Touch, Wild, Undead Controlling). The EnchantmentType enum has ~75 unique entries after deduplication.

Appendix B: Bonus Equivalent Quick Reference

Equiv	Weapon Cost (×2000)	Armor Cost (×1000)	Examples
+1	2,000	1,000	Flaming, Frost, Shock, Keen, Ghost Touch, Bane
+2	8,000	4,000	Holy, Unholy, Flaming Burst, Wounding, Disruption, SR 13, Animated
+3	18,000	9,000	Speed, Fort Moderate, Ghost Touch Armor, Invulnerability, SR 15, Wild
+4	32,000	16,000	Brilliant Energy, Dancing, SR 17
+5	50,000	25,000	Vorpal, Fort Heavy, Reflecting, SR 19
+6	72,000	36,000	(combinations only)
+7	98,000	49,000	(combinations only)
+8	128,000	64,000	(combinations only)
+9	162,000	81,000	(combinations only)
+10	200,000	100,000	Maximum (e.g., +5 Vorpal = +10 equiv)